SÓCIO DIGITAL

Relatório Consolidado da Frente de Ciência de Dados

Branch: feat/rag-pipeline-ingestao

Data de Emissão: 04/11/2026 às 16:21

RESPONSÁVEL
Jhessica Evelyn

CARGO OFICIAL
Cientista de Dados

GRUPO
Grupo 08 (RES-IA)

STATUS DA BRANCH
100% Sincronizada

TESTES TÉCNICOS
16/16 Testes OK

INTEGRAÇÃO TRIPLA
NER + Motor + RAG

Resumo. ESCOPO E OBJETIVOS DA FRENTE

Este documento sintetiza todas as entregas técnicas realizadas na branch feat/rag-pipeline-ingestao, sob responsabilidade de Jhessica Evelyn (Ciência de Dados). O foco central foi estabelecer a base de conhecimento factual em legislação tributária brasileira (LC 123/2006, CGSN 140/2018 e Manuais da Receita Federal), eliminando alucinações por meio de chunking hierárquico, busca híbrida, extração de entidades fiscais e integração com o motor de cálculo determinístico.

Arquivos Entregues e Validados na Branch

Arquivo	Responsabilidade Técnica
src/rag/chunker.py	Chunking Hierárquico Jurídico com Context Injection.
src/rag/vector_store.py	Vector Store SQLite híbrido (denso 70% + léxico 30%).
src/rag/ingest.py	Pipeline de ingestão e geração do manifesto vetorial.
src/rag/entity_extractor.py	NER Fiscal, Classificação de Intenção e Query Expansion.
src/fiscal/calculator.py	Motor Determinístico de Alíquotas e Fator R (Lucas).
test_pipeline.py	Validação dos 4 casos canônicos de RAG (100% de sucesso).
test_fiscal.py	Validação dos 6 casos de cálculo determinístico (100% de sucesso).
test_entity_extractor.py	Validação do NER e da Integração Tripla (100% de sucesso).

1. HISTÓRICO CRONOLÓGICO DE COMMITS DA BRANCH

A trajetória de versionamento da branch divide-se em duas etapas: a consolidação documental e de requisitos do Challenge 2 e a implementação rigorosa dos pipelines de IA e dados fiscais.

Bloco A: Fundação Documental e Alinhamento de Escopo

eaff3c2	DOC	Inicializa documentação e GitHub Pages Docsify	Jhessica | 03/11
46e9a86	TEAM	Atualiza matriz de papéis com Jhessica em Ciência de Da...	Jhessica | 03/11
9bb2732	DOC	Adiciona definição do projeto e guiding questions	Jhessica | 03/11
fdf4a5e	PDF	Formaliza projeto Sócio Digital e gera PDF entregáveis ...	Jhessica | 03/11
66c3911	PDF	Integra requisitos completos e gera versão 2 do PDF	Jhessica | 03/11
ed9713a	PDF	Gera versão final oficial dos entregáveis do Challenge ...	Jhessica | 03/11

Bloco B: Implementação de Ciência de Dados e Engenharia Fiscal

a64f21b	RAG	feat(rag): chunking hierárquico e busca híbrida	Jhessica | 03/11
3a12950	FIX	Fix: robust RAG vector fallback para ambientes sem GPU/...	Jhessica | 03/11
1be8277	DEV	feat(fiscal): motor determinístico e ingestão de CNAEs	Jhessica | 03/11
058fb01	NER	feat(rag): extrator de entidades fiscais e integração t...	Jhessica | 04/11

2. COMPONENTE 1: CHUNKING HIERÁRQUICO JURÍDICO

Diferente do chunking ingênuo (por contagem cega de caracteres), o LegalHierarchicalChunker (src/rag/chunker.py) preserva a estrutura normativa brasileira:

- **Preservação da Delimitação Legal:** cada trecho respeita a fronteira exata de Artigos, Caputs, Parágrafos (§) e Incisos.
- **Injeção de Contexto (Context Injection):** cada chunk recebe um cabeçalho explícito [FONTE: ... | CAPÍTULO: ... | ARTIGO: ... | TEMA: ...], impedindo que parágrafos fiquem órfãos de seu contexto legal.
- **Extração Automática de Metadados:** rotulagem em tempo de ingestão de tags como 'mei', 'fator_r', 'desenquadramento', 'obrigacoes_acessorias', 'certidoes' e 'parcelamento'.
- **Volumetria da Base Ingerida:** 18 chunks especializados gerados a partir de 3 diplomas oficiais, mapeando 11 artigos e 10 tags fiscais.

3. COMPONENTE 2: VECTOR STORE HÍBRIDO COM FALLBACK

O LegalVectorStore (src/rag/vector_store.py) adota uma arquitetura híbrida de busca com resiliência total:

Mecanismo de Busca	Peso no Score	Finalidade Técnica
Similaridade Semântica Densa	70%	Embeddings multilíngues (MiniLM-L12-v2) para similaridade conceitual.
Correspondência Léxica	30%	Correspondência exata de siglas fiscais (DASN, CPEN, CND, RBT12).
Fallback Scikit-Learn	Automático	Ativado se a biblioteca sentence-transformers não estiver instalada.
Fallback Python Puro	Automático	Cálculo vetorial TF-IDF manual sem nenhuma dependência externa.

Fórmula do Score Híbrido:

Score Final = 0.70 × CosineSimilarity(query_vec, doc_vec) + 0.30 × LexicalMatchRatio(query_tokens, doc_tokens)
Essa combinação garante que siglas críticas como CPEN e DASN-SIMEI não sejam perdidas pela busca puramente semântica.

4. COMPONENTE 3: EXTRATOR DE ENTIDADES FISCAIS (NER)

O TaxEntityExtractor (src/rag/entity_extractor.py) é o cérebro que interpreta a dúvida do empreendedor e traduz linguagem natural leiga em parâmetros exatos para o motor de cálculo e para o RAG.

Entidade Fiscal	Comportamento de Extração
CNPJ	Formato padrão 00.000.000/0000-00 e 14 dígitos numéricos com validação de máscara.
CNAE	Formato 6201-5/01 e 7 dígitos, com busca semântica no banco SQLite se não houver código explícito.
Regimes e Portes	MEI, ME, EPP, Simples Nacional, Lucro Presumido e Lucro Real.
Anexos LC 123	Anexos I, II, III, IV e V (numerais romanos ou arábicos).
Valores com Contexto	Diferenciação precisa entre RBT12 (anual), faturamento mensal, folha 12m e débitos.
Percentuais	Identificação de regra do Fator R (28%), tolerância do MEI (20%) e alíquotas.
Obrigações e Certidões	DASN-SIMEI, CPEN, CND, Termo de Exclusão e Parcelamento Ordinário.
Intenções Detectadas	fator_r, desenquadramento_mei, regularizacao_pendencias, obrigacoes_acessorias, calculo_simples.

Diferencial: Query Expansion & Tag de Foco: o extrator injeta a descrição oficial do CNAE catalogado e termos jurídicos canônicos na consulta (ex.: 'Art. 18, Anexo III vs V') e define uma tag prioritária para isolar a busca vetorial apenas na partição relevante da lei.

5. INTEGRAÇÃO TRIPLA: EXTRATOR → MOTOR → RAG

Cumprindo a diretriz de arquitetura de Clara e Nasser, o LLM nunca realiza contas matemáticas livremente. O pipeline foi testado e homologado de ponta a ponta com um caso canônico de tecnologia:

Demonstração Validada no test_entity_extractor.py:

Pergunta: 'Empresa CNAE 6201-5/01 faturou R$ 300.000,00 no ano com folha de R$ 90.000,00. Como fica o imposto?'

1. Extrator (NER): identificou CNAE 6201-5/01 (Software), RBT12 = R$ 300.000,00, Folha = R$ 90.000,00, intenção = fator_r e tag_foco = fator_r.
2. Motor Determinístico (Lucas): calculou Fator R = 30.0% (≥ 28%), migrou do Anexo V para o Anexo III, determinou a alíquota efetiva de 8,08% e o DAS mensal de R$ 2.020,00 (economia de R$ 2.105,00/mês).
3. RAG Factual (Jhessica): recuperou o Art. 18, § 5º-J da LC 123/2006 como fundamentação legal exata.

Resultado: 100% de precisão matemática e fundamentação jurídica sem nenhuma alucinação.

6. RESULTADOS DAS SUÍTES DE TESTES AUTOMATIZADOS

6.1 · Pipeline de Ingestão e Recuperação RAG (test_pipeline.py)

OK	Caso 1: MEI & Desenquadramento por 20% -> Art. 18-A da LC 123/2006
OK	Caso 2: Fator R (28%) e Anexo III vs V -> Art. 18, § 5º-J da LC 123/2006
OK	Caso 3: Prazo de entrega DASN-SIMEI (31 de maio) -> Art. 106 da Res. CGSN 140
OK	Caso 4: Regularidade fiscal e certidão CPEN -> Art. 1º do Manual da Receita Federal

6.2 · Motor Determinístico Fiscal (test_fiscal.py)

OK	Ingestão do banco SQLite com 46 CNAEs e 30 faixas dos Anexos I a V
OK	Busca FTS5 e resolução de CNAEs por código e texto
OK	Cálculo Simples Nacional Comércio (Anexo I, alíquota efetiva 4,83%)
OK	Aplicação do Fator R com migração do Anexo V para o Anexo III
OK	Diagnóstico MEI e cálculo da guia fixa DAS-MEI (R$ 75,60/mês)
OK	Desenquadramento de MEI por excesso > 20% com efeito retroativo

6.3 · NER Fiscal e Integração Tripla (test_entity_extractor.py)

OK	Caso 1: MEI excesso de faturamento e limites (R$ 105.000 vs R$ 81.000)
OK	Caso 2: CNAE 6201-5/01 e cálculo automatizado de Fator R (30%)
OK	Caso 3: Obrigação acessória DASN-SIMEI e penalidades
OK	Caso 4: Regularização, termo de exclusão e certidão CPEN
OK	Caso 5: CNPJ formatado e simulação Simples Nacional
OK	Caso 6: Integração Tripla de ponta a ponta (NER + Motor + RAG)

7. O PASSO FALTANTE: CAMADA DE GERAÇÃO E GUARDRAILS

Com a Recuperação (R) e a Extração Estruturada (NER) concluídas e testadas, o passo faltante para fechar o ciclo de IA é a Geração Aumentada (o 'G' do RAG). Este módulo será implementado em src/rag/generator.py:

- **1. Síntese em até 3 Passos (RF07):** transformar diagnósticos complexos em planos de ação executáveis (Passo 1: Diagnóstico; Passo 2: Procedimento/Guia; Passo 3: Comprovação).
- **2. Citação Mandatória de Fontes (RF08):** toda resposta gerada deve citar a norma, o artigo e o parágrafo exatos recuperados pelo RAG.
- **3. Abstenção Proativa (RNF04):** se o contexto não for suficiente, o modelo deve declarar que não possui fundamentação oficial e orientar o contato com o contador habilitado (CRC).
- **4. Fronteira Ética e Disclaimer:** garantia de que a ferramenta atua como copiloto informativo de letramento fiscal sem prestar assessoria contábil privativa.
- **5. Integração com FastAPI:** exposição de interface simples rag_service.answer_query(...) para consumo imediato nos endpoints desenvolvidos por Lucas.

Metas de Qualidade e MLOps Alinhadas com Ingrid:

· Acurácia Global: ≥ 85% com 0% de erros críticos de enquadramento tributário.
· Ancoragem em Fontes Oficiais: ≥ 95% das orientações fundamentadas.
· Latência Máxima: < 3 segundos na API e < 10 segundos em consultas RAG complexas (RNF08).
· Rastreabilidade: integração de métricas de Faithfulness e Context Precision no MLflow.

Conclusão da Avaliação de Ciência de Dados

A branch feat/rag-pipeline-ingestao está estável, com 100% dos testes aprovados e pronta para a Camada de Geração.